Progetto Laboratorio di Microcontrollori

SENSORE DI SCARICHE ELETTROSTATICHE

Group Number : A - 03

Working Team :

Giulia Emili 364565
Roberto Matteucci 365604
Francesca Moretti 369131
Luis Alessandro Siancas Curi 370778
Lorenzo Trinati 367257



A.D. 1308
unipg
UNIVERSITÀ DEGLI STUDI
DI PERUGIA

MAIN SPECIFICATIONS

L'obiettivo del progetto è monitorare e processare i dati relativi alle scariche elettrostatiche, per poi stampare in tempo reale sul display seriale il numero di scariche rilevate e la loro distanza stimata dall'apparecchio.

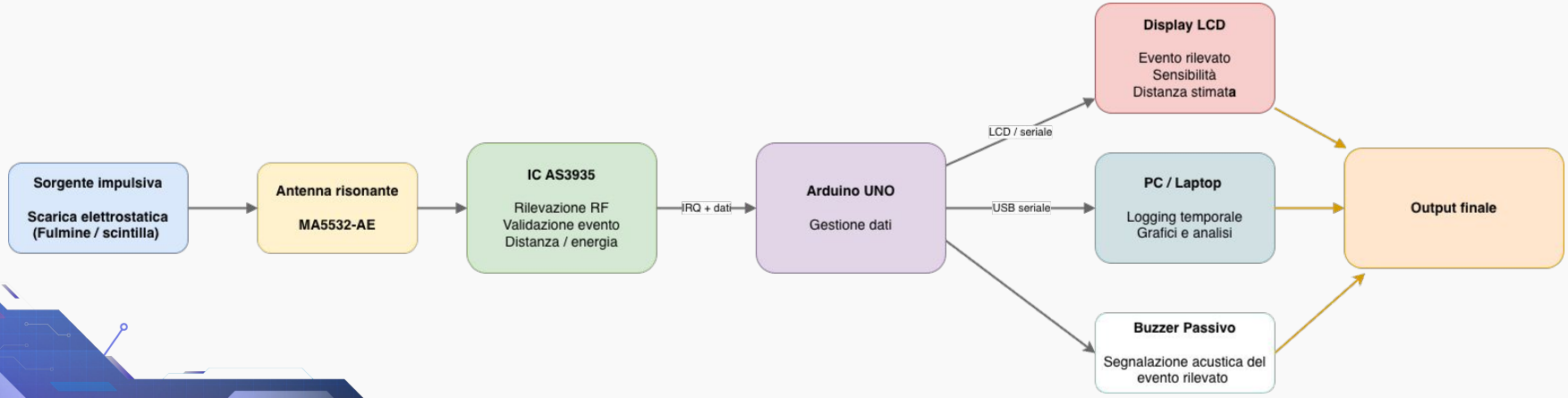
STATE-OF-THE-ART

A differenza dei sistemi di rilevamento professionali, come i ricevitori VLF (Very Low Frequency) ad alta precisione o i sensori satellitari LI (Lightning Imager), il nostro progetto utilizza un sensore economico. Questa scelta comporta una precisione minore e una certa sensibilità alle interferenze, ma risulta comunque funzionale per la realizzazione del progetto.

METHODS

1. Acquisizione del segnale RF tramite antenna risonante collegata all'AS3935.
2. Configurazione del sensore via Arduino, con regolazione della sensibilità.
3. Gestione dell'interrupt IRQ per identificare l'evento rilevato.
4. Lettura dei registri di distanza ed energia per gli eventi classificati come lightning.
5. Visualizzazione sul display seriale del PC per logging e grafici temporali.

PRELIMINARY BLOCK DIAGRAM OF THE SYSTEM



MATERIALS

- Arduino UNO;
- Breadboard & jumper wires;
- Active Buzzer
- 10 kΩ pull-up resistor (button line)
- AS3935 Franklin Lightning Sensor IC (ams AG);
- 16×2 LCD Display (HD44780-compatible, 4-bit mode);
- Push-button (momentary, normally open)
- Sfera di fulmini
- Accendino piezoelettrico

SENSORS/ACTUATORS DESCRIPTION AND AVAILABILITY

L'AS3935 é un IC progettato per rilevare la presenza di attività elettrostatica potenzialmente pericolosa nelle vicinanze, e per fornire una stima della distanza dal centro della tempesta.
Per garantire la massima precisione, il sensore sfrutta un algoritmo integrato che analizza la forma d'onda del segnale ricevuto, riuscendo a identificare e scartare i falsi postivi causati da interferenzee disturbi artificiali. L' AS3935 è in grado di segnalare tempestivamente al microcontrollore eventuali livelli eccessivi di rumore di fondo o interferenze. La programmazione e la comunicazione con il sensore possono avvenire sfruttando lo standard SPI oppure l'interfaccia I^2C, scegliendo quest'ultima soluzione, l'utente ha la flessibilità di configurare il dispositivo su tre diversi indirizzi.

PROGRAM STRUCTURE

Setup: Inizializzazione bus I2C/SPI, calibrazione della frequenza di risonanza dell'antenna (per l'AS3935).

Loop: Arduino rimane in idle o gestisce altre funzioni finché non riceve un interrupt dal sensore.

FLOW CHART

1. Inizio: calibrazione;
2. Attesa di un interrupt;
3. Lettura registro "Interrupt" dell'AS3935;
4. Classificazione: fulmine, disturbo o rumore eccessivo?;
5. Se è un fulmine: calcola la distanza e riportala sul display;

NEXT STEPS

01

Studio del sensore AS3935 e delle funzionalità

02

Prototipo del circuito

03

Scrittura codice

04

Prova funzionamento e rifiniture

01

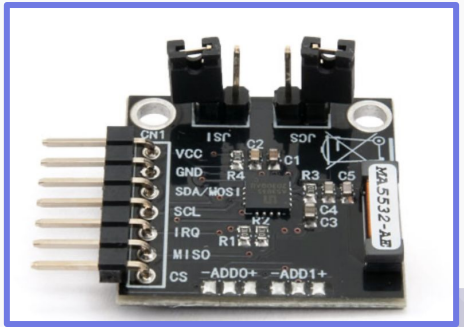
SENSORE AS3935

SENSORE AS3935

Il chip AS3935 presenta i seguenti pin:

- IRQ: l'uscita tramite la quale vengono inviati gli Interrupt ad Arduino;
- MISO: l'uscita tramite la quale vengono inviati ad Arduino i dati sui fulmini rilevati;
- MOSI: l'uscita tramite la quale Arduino comunica al chip messaggi di impostazione o conferma;
- CS: permette di abilitare l'interazione sul bus SPI, dato che sull'SPI possono affacciarsi più dispositivi, questo pin permette di evitare conflitti di comunicazione;
- SCL: segnale di sincronizzazione per la comunicazione; comunicazione SPI.

In quanto alla comunicazione seriale il chip è abilitato alle modalità SPI e I2C, le quali possono essere selezionate tramite il jumper JSI:



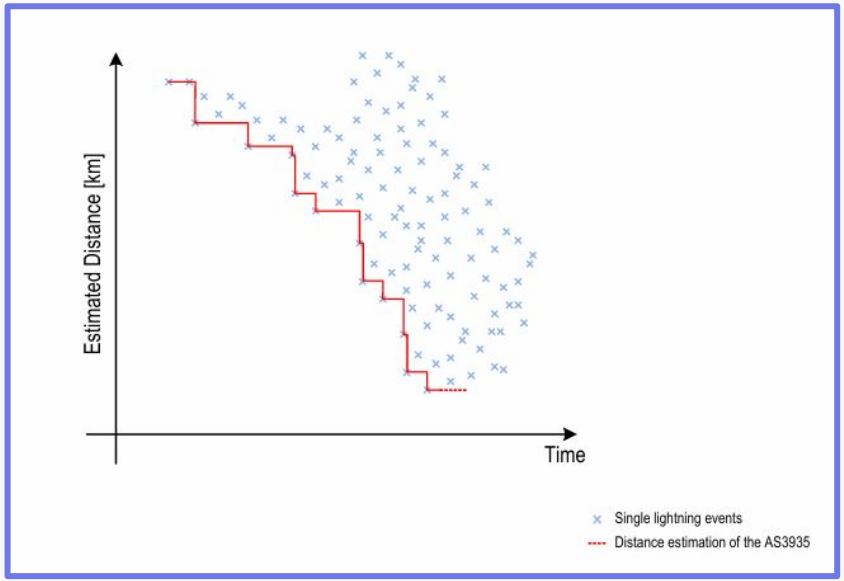
- SPI: il jumper è chiuso;
- I2C: il jumper è aperto;

Il jumper JSC permette di stabilire qual è l'indirizzo associato al sensore laddove sia abilitata la comunicazione SPI.

SENSORE AS3935

Algoritmo di rilevazione della distanza

Il sensore non si limita a rilevare il fulmine, ma elabora la distanza dalla "testa" della tempesta attraverso un sistema **statistico hardwired** (integrato nel chip).



Come funziona il calcolo:

- ➔ Memoria Interna;
- ➔ Correlazione e Look-Up Table;
- ➔ Gestione Dinamica;

Dati in Uscita:

- ➔ Registro REG0x07;
- ➔ Aggiornamento Continuo;

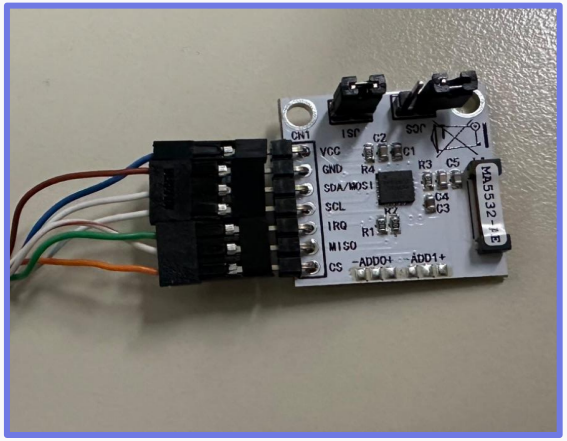
SENSORE AS3935

Antenna Tuning

Per intercettare i segnali elettromagnetici dei fulmini, l'AS3935 utilizza un'antenna (MA5532-AE) ad anello basata su un **risonatore LC parallelo**.

Questo componente funge da orecchio del sistema e deve essere tarato con estrema precisione per isolare le frequenze generate dalle scariche atmosferiche rispetto ai rumori di fondo.





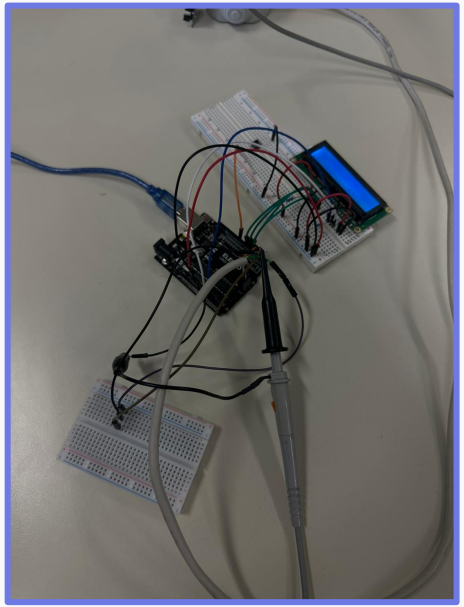
Punti chiave della calibrazione:

- ➔ Frequenza di target;
- ➔ Precisione del ±3,5%;
- ➔ Ottimizzazione;

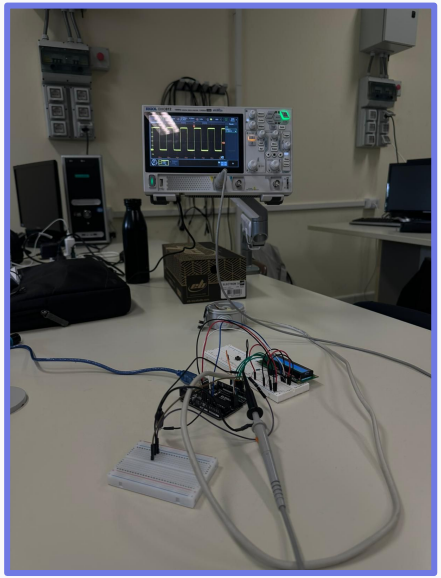
SENSORE AS3935

Test con oscilloscopio

Per garantire l'affidabilità del sistema, abbiamo effettuato una verifica strumentale della frequenza di risonanza utilizzando un oscilloscopio digitale.
Questa fase è stata cruciale per confermare che la sintonizzazione dell'antenna LC fosse centrata quanto più possibile sui 500 kHz, minimizzando gli errori di rilevamento.

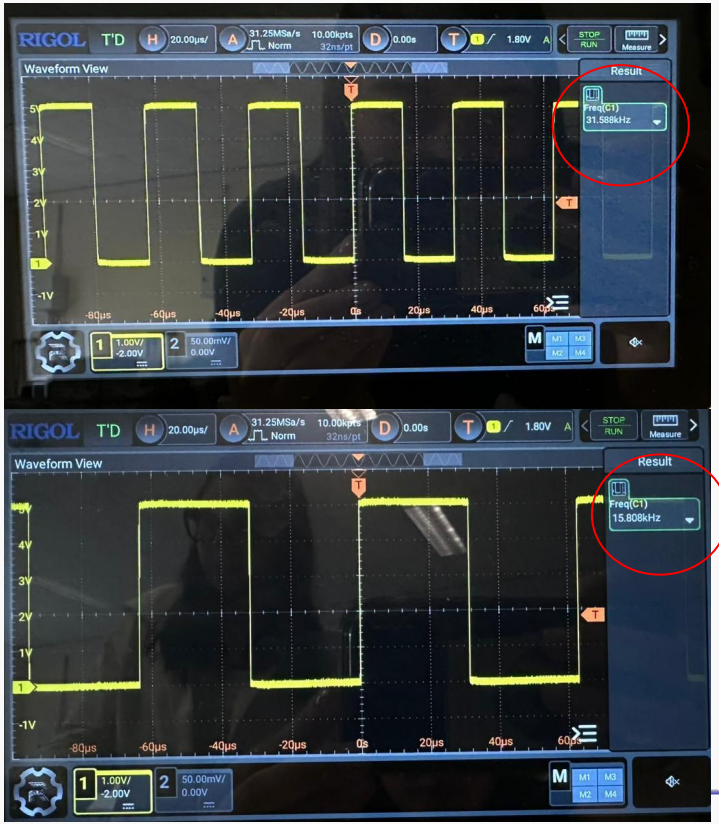






SENSORE AS3935

Test con oscilloscopio



I condensatori interni al circuito integrato (IC) possono essere sintonizzati per far rientrare l'antenna nel range corretto di rilevamento. Questa calibrazione si effettua tramite la funzione **tuneCap(int pF)**, che accetta valori interi da **8 pF a 120 pF**, con una risoluzione (passo) di **8 pF**.
La frequenza di risonanza effettiva si ottiene moltiplicando il valore misurato dall'oscilloscopio per un fattore di divisione/qualità.

Questo parametro è impostato di default a **16**, ma può essere modificato a piacimento:

Configurazione standard (immagine in alto): utilizzando il fattore di default pari a **16**, la frequenza rilevata è di **31.588 kHz**.

Configurazione modificata (immagine in basso): raddoppiando il fattore a **32**, la frequenza rilevata si dimezza a **15.808 kHz**.

SENSORE AS3935

Test con oscilloscopio



Prima della taratura:
frequenza rilevata * fattore di qualità =
33.238 kHz * 16 ≃ 532 kHz
Ben oltre il range di frequenza.

Dopo la taratura:
frequenza rilevata * fattore di qualità =
31.588 kHz * 16 ≃ 505 kHz
Perfettamente nel range di frequenza.

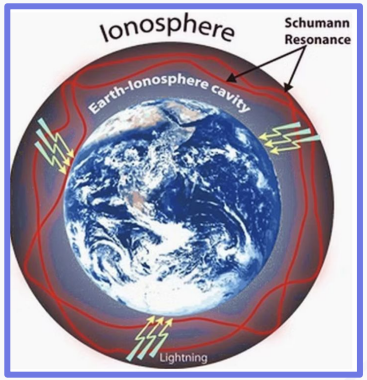


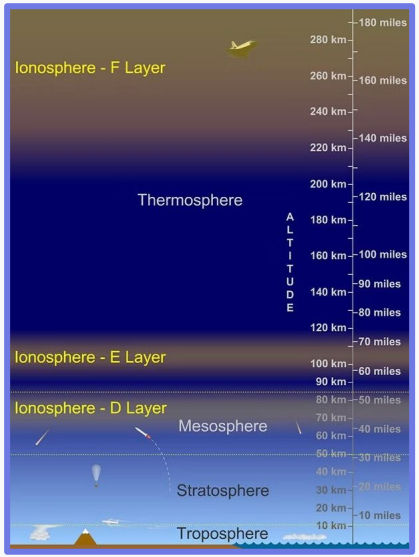
SENSORE AS3935

Limite del sensore e risonanza di Shumann

Sebbene l'AS3935 sia uno strumento avanzato per la prototipazione, la sua efficacia è limitata rispetto ai sistemi di monitoraggio globale che sfruttano le Risonanze di Schumann.

- ➔ Gap di frequenza e portata;
- ➔ Inquinamento Elettromagnetico (EMI);
- ➔ Dimensioni dell'Hardware;





BRMI
Bioregulatory Medicine Institute
Supporting the Science of Self-Healing

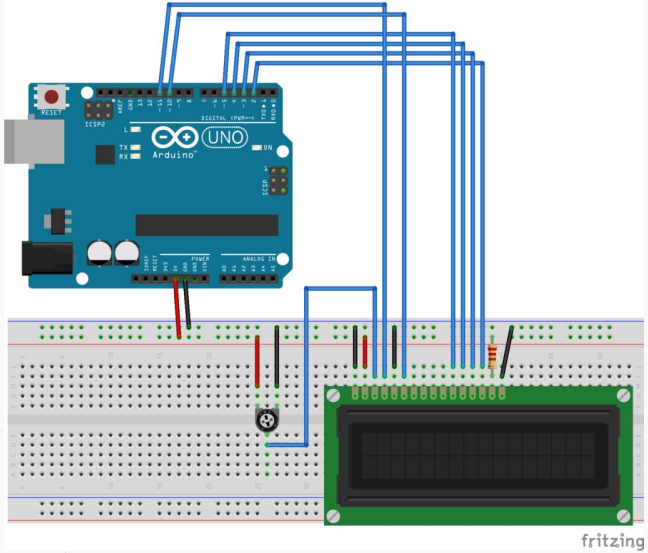
WIKIPEDIA 25

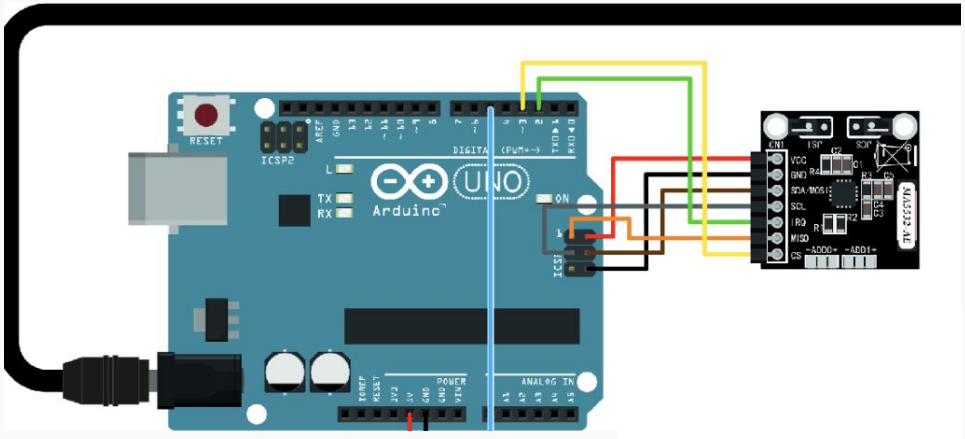
02

PROTOTIPO DEL CIRCUITO

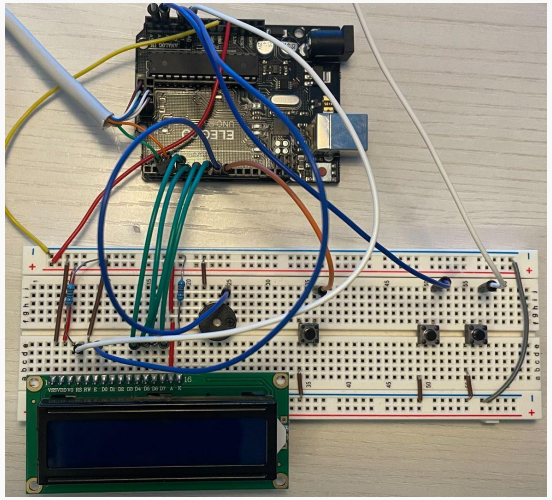
PROTOTIPO DEL CIRCUITO

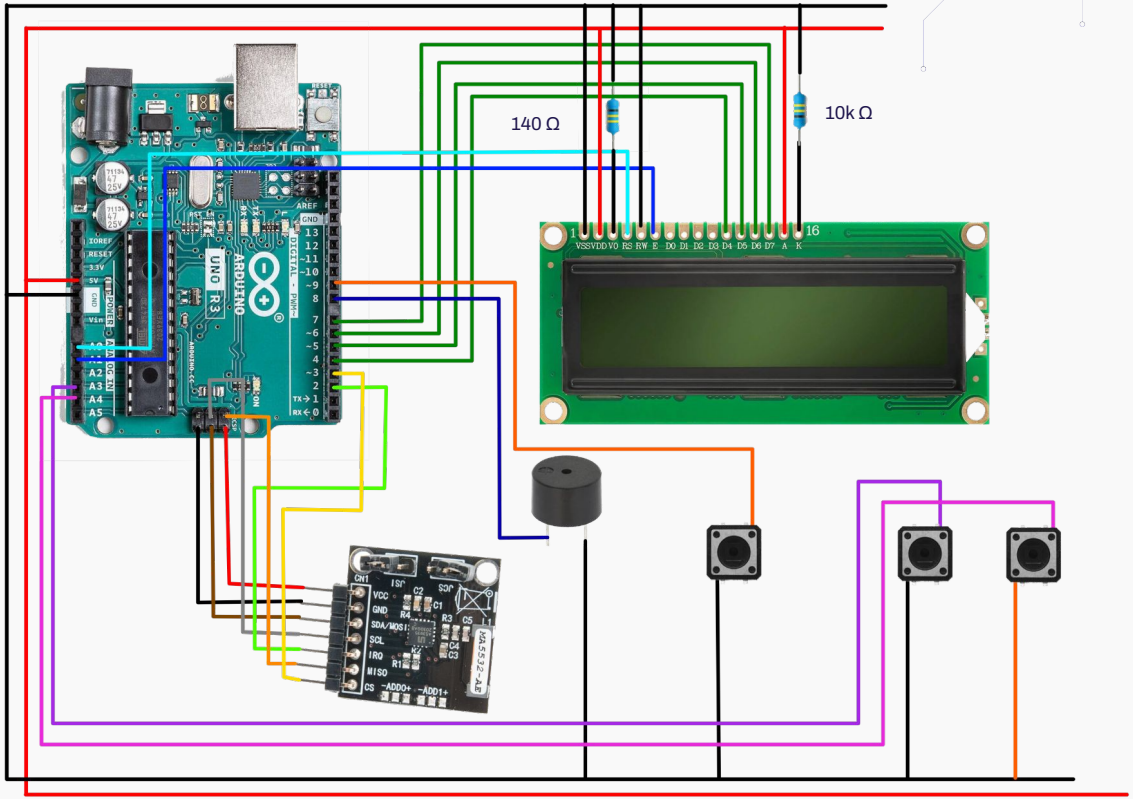
Per la configurazione del circuito, abbiamo analizzato la documentazione tecnica e gli schemi di riferimento disponibili online per l'interfaccia tra il sensore e il microcontrollore.
Partendo da questi modelli, abbiamo ottimizzato i cablaggi per adattarli alla nostra specifica scheda Arduino.





PROTOTIPO DEL CIRCUITO





03

SCRITTURA DEL CODICE

SCRITTURA DEL CODICE

Architettura Hardware

Il cuore tecnologico è il sensore AS3935, che dialoga con Arduino tramite protocollo **SPI** e utilizza il pin **IRQ** per notificare gli eventi elettromagnetici.

L'interfaccia utente prevede:

- ➔ display LCD gestito a 6 pin;
- ➔ buzzer per i segnali acustici;
- ➔ 3 pulsanti per il controllo

Dal punto di vista logico, abbiamo implementato due stati gestiti dal **pulsante ON**:

Quando il sistema è attivo: monitora l'ambiente in tempo reale.

Quando è spento: smette di elaborare i fulmini ma lascia la memoria accessibile per la consultazione.

```
57  // ============================================================
58  //  LIBRARIES
59  // ============================================================
60  #include <SPI.h>
61  #include <LiquidCrystal.h>
62  #include "SparkFun_AS3935.h"
63
64
65  // ============================================================
66  //  HARDWARE
67  // ============================================================
68  LiquidCrystal lcd(A0, A1, 4, 5, 6, 7);
69
70  const int BTN_ON   = 9;
71  const int BTN_NEXT = A4;
72  const int BTN_PREV = A3;
73
74  const int IRQ_PIN    = 2;
75  const int CS_PIN     = 3;
76  const int BUZZER_PIN = 8;
77
78  SparkFun_AS3935 lightning;
```

SCRITTURA DEL CODICE

Impostazione dei parametri del sensore

Definiamo le costanti che ci permettono di tarare il sensore. Questo è l'unico modo di modificare i registri interni al sensore, ovvero tramite i parametri alle **righe 90-94**.

In particolare, **CAPACITOR_VALUE** è il parametro da inserire nella funzione **tuneCap(int pF)** per la sintonizzazione dell'antenna.

```
81  // ====================================
82  //  AS3935 SETTINGS
83  // ====================================
84  #define INDOOR          0x12
85  #define OUTDOOR         0x0E
86  #define LIGHTNING_INT   0x08
87  #define DISTURBER_INT   0x04
88  #define NOISE_INT       0x01
89
90  const byte CAPACITOR_VALUE  = 120;
91  const byte NOISE_FLOOR      = 7;
92  const byte WATCHDOG_VAL     = 7;
93  const byte SPIKE_REJECTION  = 2;
94  const byte LIGHTNING_THRESH = 1;
```

SCRITTURA DEL CODICE
Definizione delle variabili

Definiamo una serie di variabili, tra cui:

- La struttura **EventRecord**, che memorizza distanza, energia e timestamp del evento;
- Il buffer circolare **history**, per la memorizzazione degli ultimi n fulmini rilevati;

```
97  // ============================================================
98  //  EVENT HISTORY  (circular buffer)
99  // ============================================================
100 #define HISTORY_SIZE 24
101
102 struct EventRecord {
103   String label;
104   byte distance;
105   long energy;
106   unsigned long timestamp;
107 };
108
109 EventRecord history[HISTORY_SIZE];
110 int historyCount = 0;   // number of valid entries (max HISTORY_SIZE)
111 int writeIndex = 0;   // next write slot in the circular buffer
```

SCRITTURA DEL CODICE

Definizione delle variabili

Una serie di variabili per la gestione del display, dei pulsanti e del buzzer acustico.

```
123  // ============================================================
124  //  SYSTEM STATE
125  // ============================================================
126  bool systemActive = false;
127
128  const unsigned long DEBOUNCE_MS = 50; // buttons debounce
129  const int BUZZER_FREQ = 5500; // buzzer sound
130
131  // History browse state
132  bool historyView  = false;
133  int viewIndex = 0;
134  unsigned long lastBrowseMs = 0;
135
136  const unsigned long BROWSE_TIMEOUT_MS = 3000;  // ms before browse auto-exits
137
138  // Label shown in row 0 during LIVE mode
139  const char* currentLabel = LABEL_WAITING;
140
141  //  After each sensor interrupt the lightning icon blinks ON/OFF
142  const unsigned long FLASH_INTERVAL_MS = 325;  // toggle every 125 ms (4 blinks)
143
144  bool flashActive = false;
145  bool flashState = false;
146  unsigned long flashStart = 0;
147  unsigned long lastFlashToggle = 0;
148
149  // Live label timer
150  const unsigned long LABEL_SHOW_MS = 2000;
151  bool labelActive = false;
152  unsigned long labelStart = 0;
```

SCRITTURA DEL CODICE

Funzione Setup()

La **funzione setup()** richiama la funzione **initAS3935()**, che si occupa di inizializzare tutti i parametri definiti precedentemente.

Viene inoltre richiamata la funzione **updateDisplay()**, di cui discuteremo più avanti.

```
180  // ============================================================
181  //  SETUP
182  // ============================================================
183  void setup() {
184
185    Serial.begin(115200);
186    Serial.println(F("[Setup] Booting..."));
187
188    pinMode(BTN_ON,   INPUT_PULLUP);
189    pinMode(BTN_NEXT, INPUT_PULLUP);
190    pinMode(BTN_PREV, INPUT_PULLUP);
191    pinMode(BUZZER_PIN, OUTPUT);
192    pinMode(IRQ_PIN, INPUT);
193
194    lcd.begin(16, 2);
195    lcd.createChar(0, lightningOff);
196    lcd.createChar(1, lightningOn);
197
198    SPI.begin();
199    initAS3935();
200
201    updateDisplay();
202    Serial.println(F("[Setup] Ready. Press ON to start monitoring."));
203  }
```

SCRITTURA DEL CODICE

Funzione Loop ()

Per garantire che il sistema non perdesse alcun fulmine e rimanesse reattivo ai comandi dell'utente, abbiamo evitato l'uso di delay bloccanti.

Sfruttando la **funzione millis()**, il loop esegue ciclicamente quattro compiti in parallelo:

- ➔ **Controllo dei pulsanti**, applicando un filtro antirimbalzo software;
- ➔ **Gestione di un timeout di 3 secondi**, per uscire dalla cronologia se l'utente non preme nulla;
- ➔ **Interrogazione del pin IRQ** del sensore;
- ➔ **Coordinamento delle animazioni** del display in background;

```
208  void loop() {
209    handleButtons();
210
211    // Auto-exit browse mode after inactivity timeout
212    if (historyView && millis() - lastBrowseMs > BROWSE_TIMEOUT_MS) {
213      historyView = false;
214      updateDisplay();
215    }
216
217    // Poll sensor only when the system is active
218    if (systemActive && digitalRead(IRQ_PIN) == HIGH) {
219      processAS3935Event();
220    }else if(!systemActive){
221      lightning.readInterruptReg();   // clear IRQ
222    }
223
224    // blink the flash icon + buzzer sound
225    if (flashActive) {
226      unsigned long now = millis();
227
228      if (now - lastFlashToggle >= FLASH_INTERVAL_MS) {
229        lastFlashToggle = now;
230        flashState = !flashState;
231        lcd.setCursor(0, 0);
232        lcd.write(flashState ? byte(1) : byte(0));
233        flashActive = false;
234
235        //buzzer sound
236        playSound(BUZZER_FREQ, 500);
237      }
238    }
239
240    // live label
241    if (labelActive && millis() - labelStart >= LABEL_SHOW_MS) {
242      labelActive  = false;
243      currentLabel = LABEL_WAITING;
244      if (!historyView) updateDisplay();
245    }
246  }
```

SCRITTURA DEL CODICE
Meccanismo di gestione degli eventi

Il pin **IRQ** diventa alto e il codice esegue la funzione **processAS3935Event()**. Rispettando le specifiche del datasheet del sensore, attendiamo 2 millisecondi per far stabilizzare il dato e successivamente interroghiamo il registro via **SPI**.

Il sensore è in grado di classificare l'evento in tre categorie:

- **Rumore;**
- **Disturbatore elettromagnetico** (disturber);
- **Fulmine reale;**

In quest'ultimo caso, andiamo a leggere anche la **distanza stimata del fronte della tempesta** e l'**energia intrinseca del fulmine** (valore adimensionale), che verranno poi inviati alla memoria e al display.

```
322 void processAS3935Event() {
323   delay(2);   // AS3935 datasheet 5.4: wait 2 ms before reading IRQ register
324   byte intVal = lightning.readInterruptReg();
325
326   const char* label = nullptr;
327   byte dist = 0;
328   long energy = 0;
329
330   if (intVal == NOISE_INT) {
331     label  = LABEL_NOISE;
332     Serial.println(F("[EVENT] Noise"));
333
334   } else if (intVal == DISTURBER_INT) {
335     label  = LABEL_DISTURBER;
336     Serial.println(F("[EVENT] Disturber"));
337     Serial.print(dist);
338
339   } else if (intVal == LIGHTNING_INT) {
340     dist   = lightning.distanceToStorm();
341     energy = lightning.lightningEnergy();
342     label  = LABEL_LIGHTNING;
343     Serial.print(F("[EVENT] Lightning! "));
344     Serial.print(dist);
345     Serial.print(F(" km | energy: "));
346     Serial.println(energy);
347
348   } else {
349     return;  // unknown interrupt — ignore
350   }
```

SCRITTURA DEL CODICE

Interfaccia

Abbiamo creato dei caratteri personalizzati per mostrare l'icona del fulmine che lampeggia ad ogni rilevamento.

Il display cambia layout a seconda dello stato:

- ➔ **Modalità live:** mostra l'attività corrente del sistema;
- ➔ **Modalità cronologia:** si attiva premendo i tasti NEXT o PREV;

```
393  void updateDisplay() {
394    lcd.clear();
395
396    // --- BROWSE mode ---
397    if (historyView && historyCount > 0) {
398      int slot = historySlot(viewIndex);
399
400      // Row 0: "[index/TOT] HH:MM:SS"
401      lcd.setCursor(0, 0);
402
403      lcd.print("[");
404      lcd.print(viewIndex + 1);
405      lcd.print("/");
406      lcd.print(historyCount);
407      lcd.print("] ");
408
409      lcd.print(formatTimestamp(history[slot].timestamp));
410
411      // Row 1: full label + distance (if available)
412      lcd.setCursor(0, 1);
413      if(history[slot].distance > 0 && history[slot].energy > 0){
414        lcd.print(history[slot].label.substring(0,4));
415
416        lcd.print(" ");
417        lcd.print(history[slot].distance);
418        lcd.print("km - ");
419
420        lcd.print(formatEnergy(history[slot].energy));
421      }else{
422        lcd.print(history[slot].label);
423      }
424      return;
425    }
```

04

PROVA FUNZIONAMENTO E RIFINITURE

PROVA FUNZIONAMENTO

Verificare il corretto funzionamento del sensore ha rappresentato decisamente la sfida più grande del progetto.





Per farlo, abbiamo utilizzato tre diversi strumenti di verifica:

- ➔ **Accendino piezoelettrico;**
- ➔ **Sfera al plasma;**
- ➔ **Una tempesta vera e propria;**

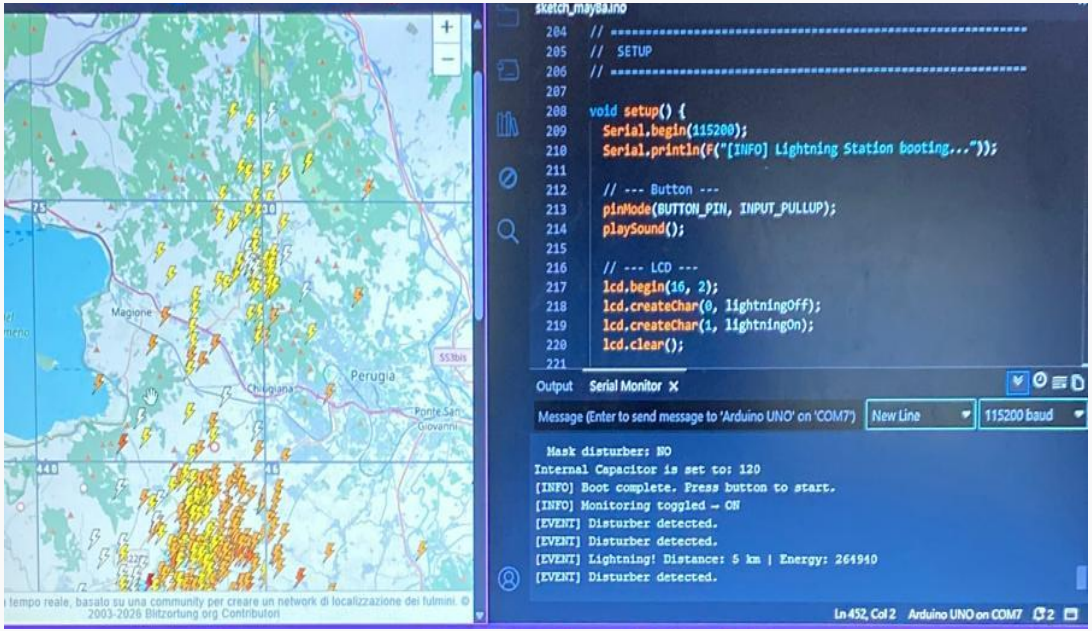
La sperimentazione con i primi due oggetti verrà dimostrata direttamente in laboratorio.

A questo proposito, è necessaria una piccola precisazione: il sensore percepisce (giustamente) l'accendino e la sfera al plasma come **DISTURBI** e non come **FULMINI**. Per scopi didattici e seguendo il consiglio del professore, abbiamo deciso di ignorare questa classificazione.

Fatta questa considerazione, aggiungiamo che in questo modo il sensore non è in grado di rilevare distanza e potenza (appunto perché non rileva un fulmine ma un disturbo), ecco il motivo per il quale abbiamo deciso di non mostrare a schermo queste due misure durante la dimostrazione.

Abbiamo considerato terminato il progetto una volta constatato che il sensore reagisse a una delle due sollecitazioni (e quindi che non rilevasse eventi senza motivo).

PROVA FUNZIONAMENTO



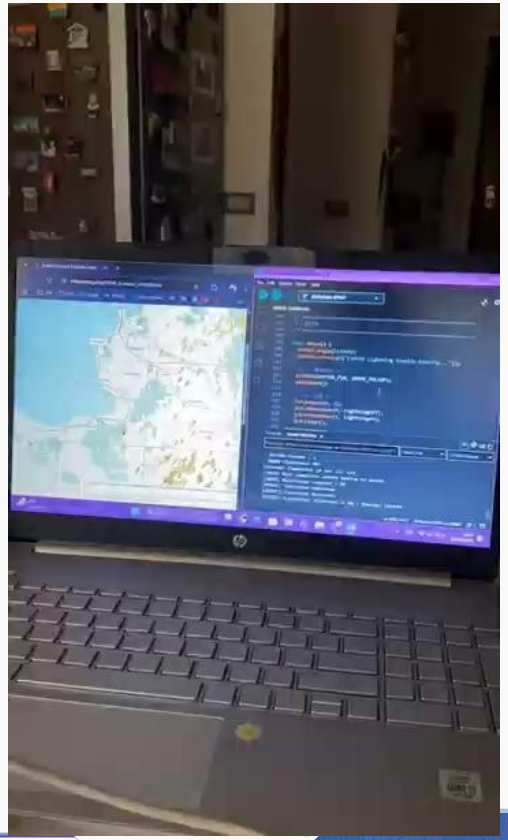
mappa dei fulmini in tempo reale

```
[EVENT] Lightning! Distance: 20 km | Energy: 13320
[EVENT] Disturber detected.
[EVENT] Lightning! Distance: 1 km | Energy: 88777
[EVENT] Lightning! Distance: 1 km | Energy: 158204
```

```
Internal Capacitor is set to: 120
[INFO] Boot complete. Press button to start.
[INFO] Monitoring toggled → ON
[EVENT] Lightning! Distance: 20 km | Energy: 11151
```

```
Internal Capacitor is set to: 120
[INFO] Boot complete. Press button to start.
[INFO] Monitoring toggled → ON
[EVENT] Lightning! Distance: 17 km | Energy: 23332
```

PROVA FUNZIONAMENTO



GRAZIE
DELL'ATTENZIONE

Group Number : A - 03

Working Team :

Giulia Emili 364565
Roberto Matteucci 365604
Francesca Moretti 369131
Luis Alessandro Siancas Curi 370778
Lorenzo Trinati 367257

A.D. 1308
unipg
UNIVERSITÀ DEGLI STUDI
DI PERUGIA